



SQL Mastery

PJ'S ACADEMY

Complete Course Book

SQL Mastery — PJ's Academy

From "what is a database?" to cracking SQL interviews — with 100 practice problems solved in BOTH standard SQL and Snowflake, fully explained.

SQL is the #1 most-requested skill in every data job. This course assumes **zero knowledge** and takes you to interview-ready, with a 100-problem bank (LeetCode-style) where every solution is commented line by line.

Who This Is For

- Complete beginners (we start from "a table is like a spreadsheet")
- Anyone prepping for data analyst / engineer / scientist interviews
- People who "kind of know SQL" but freeze on window functions and joins

Course Structure

Part	Topic
1	Databases & SELECT basics
2	Filtering, sorting, operators
3	Aggregations & GROUP BY
4	JOINS (the big one)
5	Subqueries & CTEs
6	Window functions
7	Advanced: pivots, recursion, performance
8	100 Practice Problems (Easy → Hard)

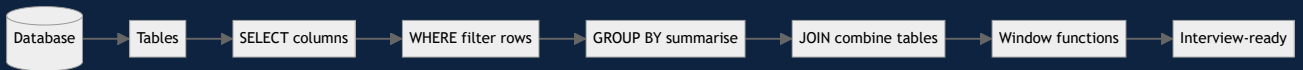
The Practice Bank — 100 Problems

The heart of this course. LeetCode-style problems, each with: - 📄 A clear problem statement + sample data - ✅ A **standard SQL** solution (works in MySQL/Postgres) - ❄️ A **Snowflake** solution (showing Snowflake-specific idioms like `QUALIFY`) - 💬 **Line-by-line explanation in comments** — so you never get stuck

Tier	Count	File
🟢 Easy	35	easy problems
🟡 Medium	40	medium problems
🔴 Hard	25	hard problems

Practice on the **shared schema** — one set of tables used across all problems.

Learn Visually



💡 How To Use This Course

1. Read a Part, run every example.
2. Do the matching practice problems (Easy → Medium → Hard).
3. **Cover the solution, try first, then compare.** That struggle is where learning happens.
4. Redo every problem you missed a week later.

🏆 Outcome

Finish this and you can walk into a SQL interview, handle window functions and multi-table joins calmly, and write clean, correct queries in both standard SQL and Snowflake.

Course: 📖 *SQL Mastery* — [PJ's Academy](#) · hello@pjsacademy.com

Module 01 — Databases & SELECT Basics

Start from zero: what a database is, and how to ask it for data. By the end you'll confidently read any table.

1.1 What Is a Database?

A **database** is an organised collection of data. The kind we query with SQL is a **relational database** — data lives in **tables**, which are just like spreadsheets:

- A **table** = a grid (e.g., `employees`).
- A **row** = one record (one employee).
- A **column** = one attribute (name, salary, hire_date).
- Each column has a **type** (text, number, date).

employees

emp_id	name	dept_id	salary	← columns
1	Asha	1	180000	← a row
2	Ravi	1	120000	

SQL (Structured Query Language) is how we talk to the database — ask questions, get answers.

1.2 The Sample Database (used all course)

We use 5 related tables: `employees` , `departments` , `customers` , `products` , `orders` . Run the `schema` once to create them.

Relationships: - Each employee belongs to a department (`employees.dept_id → departments.dept_id`). - Each order belongs to a customer and a product.

1.3 SELECT — Asking for Data

The `SELECT` statement retrieves rows. The simplest form:

```
SELECT * FROM employees;
```

- `SELECT` = "give me"
- `*` = "all columns"
- `FROM employees` = "from the employees table"

1.4 Choosing Columns

`*` is fine for exploring, but in real code you name the columns you want — it's clearer and faster:

```
SELECT name, salary FROM employees;
```

Order the columns however you like:

```
SELECT salary, name FROM employees; -- salary first
```

1.5 Aliases — Renaming Output

Give a column a friendlier name in the result with `AS` :

```
SELECT name AS employee_name,  
       salary AS monthly_salary  
FROM employees;
```

`AS` is optional (`salary monthly_salary` works) but always write it — clarity wins.

1.6 Calculated Columns

You can compute new columns on the fly:

```
SELECT name,  
       salary AS monthly,  
       salary * 12 AS annual      -- arithmetic  
FROM employees;
```

Combine text with concatenation (`||` in standard SQL / Snowflake):

```
SELECT name || ' works in dept ' || dept_id AS description  
FROM employees;
```

1.7 DISTINCT — Unique Values

Remove duplicate rows from the result:

```
SELECT DISTINCT dept_id FROM employees;  -- each department once
```

1.8 LIMIT — Just a Few Rows

Return only the first N rows (great for peeking at big tables):

```
SELECT * FROM employees LIMIT 5;
```

❄ Snowflake also supports `FETCH FIRST 5 ROWS ONLY` .

1.9 Comments — Notes in Your SQL

```
-- This is a single-line comment  
SELECT name      -- you can comment at the end of a line too  
FROM employees;  
/* This is a  
   multi-line comment */
```

1.10 Reading a Query (order it runs)

You write `SELECT` first, but the database runs `FROM` first (get the table), then `SELECT` (pick columns). This matters more later with `WHERE` and `GROUP BY` — keep it in mind.

✅ Key Takeaways

1. A relational database stores data in **tables** (rows × columns), each column has a type.
2. `SELECT columns FROM table` retrieves data; `*` means all columns.
3. Use `AS` to **alias** (rename) output columns.
4. You can compute **calculated columns** (`salary * 12`) and concatenate text (`||`).
5. `DISTINCT` removes duplicates; `LIMIT` caps the number of rows.
6. The database runs `FROM` before `SELECT` .

Exercises

1. Select all columns from `products` .
2. Select just `name` and `salary` from `employees` , salary first.
3. Show each product's `name` and its price × 1.18 (with GST) as `price_with_gst` .
4. List the distinct `city` values from `customers` .
5. Show the first 3 rows of `orders` .

Next: [Module 02 — Filtering, Sorting & Operators](#) →

Module 02 — Filtering, Sorting & Operators

Rarely do you want *all* rows. This module is about asking for exactly the rows you need, in the order you want.

2.1 WHERE — Keeping Only the Rows You Want

`WHERE` filters rows by a condition. Only rows where the condition is **TRUE** are returned.

```
SELECT name, salary FROM employees
WHERE salary > 100000;
```

`WHERE` runs after `FROM` but before `SELECT` picks columns — it decides which rows survive.

2.2 Comparison Operators

Operator	Meaning
<code>=</code>	equal
<code><></code> or <code>!=</code>	not equal
<code>></code> <code><</code> <code>>=</code> <code><=</code>	greater/less than (or equal)

```
SELECT name FROM employees WHERE dept_id = 1;
SELECT name FROM employees WHERE salary >= 90000;
SELECT name FROM employees WHERE dept_id <> 2;
```

2.3 Combining Conditions: AND, OR, NOT

```
-- Both must be true
SELECT name FROM employees WHERE dept_id = 1 AND salary > 100000;
-- Either can be true
SELECT name FROM employees WHERE dept_id = 1 OR dept_id = 3;
-- Negate
SELECT name FROM employees WHERE NOT dept_id = 2;
```

⚠ `AND` binds tighter than `OR`. Use parentheses to be explicit: `WHERE (dept_id = 1 OR dept_id = 2) AND salary > 90000`.

2.4 BETWEEN — Ranges

Inclusive on both ends:

```
SELECT name, salary FROM employees
WHERE salary BETWEEN 80000 AND 120000; -- 80000 and 120000 included
```

2.5 IN — Match a List

Cleaner than many `OR` s:

```
SELECT name FROM employees WHERE dept_id IN (1, 3);
-- same as: WHERE dept_id = 1 OR dept_id = 3
SELECT name FROM employees WHERE dept_id NOT IN (2, 4);
```

2.6 LIKE — Pattern Matching

For partial text matches. Wildcards: - `%` = any sequence of characters (including none) - `_` = exactly one character

```
SELECT name FROM employees WHERE name LIKE 'A%'; -- starts with A
SELECT name FROM employees WHERE name LIKE '%a'; -- ends with a
SELECT name FROM employees WHERE name LIKE '%in%'; -- contains "in"
SELECT name FROM employees WHERE name LIKE '_a%'; -- 2nd letter is a
```

❄ Snowflake: `ILIKE` is case-insensitive (`WHERE name ILIKE 'a%'`).

2.7 NULL — the Absence of a Value

`NULL` means "unknown / no value" — it is **not** zero or empty string. You **cannot** compare it with `=`:

```
-- WRONG: returns nothing, ever
SELECT name FROM employees WHERE email = NULL;
-- RIGHT:
SELECT name FROM employees WHERE email IS NULL;
SELECT name FROM employees WHERE email IS NOT NULL;
```

⚠ Any comparison with `NULL` yields `UNKNOWN` (not `TRUE`), so those rows are excluded. This trips up everyone at first — remember `IS NULL` / `IS NOT NULL`.

2.8 ORDER BY — Sorting Results

```
SELECT name, salary FROM employees
ORDER BY salary DESC; -- highest first; ASC (default) = lowest first
```

Sort by multiple columns (tie-breakers):

```
SELECT name, dept_id, salary FROM employees
ORDER BY dept_id ASC, salary DESC; -- by dept, then salary within dept
```

You can order by a column you didn't select, or by position (`ORDER BY 2` = 2nd selected column — handy but less readable).

2.9 LIMIT with ORDER BY — Top-N

The classic "top N" pattern: sort, then limit.

```
-- Top 3 highest-paid employees
SELECT name, salary FROM employees
ORDER BY salary DESC
LIMIT 3;
```

2.10 Order of Operations (mental model)


```
FROM      → get the table
WHERE     → filter rows
SELECT    → pick/compute columns
ORDER BY  → sort
LIMIT     → cap rows
```

This is why you can't use a `SELECT` alias in `WHERE` (`WHERE` runs first) but often can in `ORDER BY` (which runs after `SELECT`).

✓ Key Takeaways

1. `WHERE` filters rows using comparison operators (`=` , `<>` , `>` , ...).
2. Combine conditions with `AND` / `OR` / `NOT` ; use parentheses to control precedence.
3. `BETWEEN` (ranges), `IN` (lists), `LIKE` (patterns with `%` and `_`).
4. `NULL` is unknown — use `IS NULL` / `IS NOT NULL` , never `= NULL` .
5. `ORDER BY col DESC/ASC` sorts; add columns for tie-breakers.
6. Sort + `LIMIT` = the top-N pattern. Remember the `FROM`→`WHERE`→`SELECT`→`ORDER BY`→`LIMIT` order.

🏆 Exercises

1. List employees earning more than 90,000, highest salary first.
2. Find employees in departments 1 or 4 (use `IN`).
3. Find employees whose name contains the letter "a".
4. Find employees with no email (`IS NULL`).
5. Show the 2 cheapest products.
6. List employees hired between '2021-01-01' and '2021-12-31' (use `BETWEEN`).

Next: [Module 03 — Aggregations & GROUP BY](#) →

Module 03 — Aggregations & GROUP BY

Going from raw rows to *summaries*: totals, averages, counts. This is where SQL starts answering business questions.

3.1 Aggregate Functions — Collapsing Many Rows into One

An aggregate takes many rows and returns a single value:

Function	Returns
COUNT(*)	number of rows
COUNT(col)	number of non-NULL values in col
SUM(col)	total
AVG(col)	average
MIN(col) / MAX(col)	smallest / largest

```
SELECT COUNT(*) AS total_employees,
       AVG(salary) AS avg_salary,
       MAX(salary) AS highest,
       MIN(salary) AS lowest,
       SUM(salary) AS total_payroll
FROM employees;
```

3.2 COUNT(*) vs COUNT(column) — a Classic Gotcha

```
SELECT COUNT(*) AS all_rows,      -- counts every row
       COUNT(email) AS with_email -- counts only non-NULL emails
FROM employees;
```

COUNT(*) counts rows; COUNT(col) ignores NULLs. Interviewers love this distinction.

3.3 GROUP BY — Aggregates Per Something

Without GROUP BY, an aggregate summarises the whole table. With it, you get **one row per group**:

```
-- Average salary PER department
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id;
```

Rule of thumb: whenever you say "per X" (per department, per month, per customer), X goes in GROUP BY.

```
Before GROUP BY (rows)      After GROUP BY dept_id
dept 1, 180000              dept 1 → avg 136666
dept 1, 120000              dept 2 → avg 82500
dept 1, 110000              dept 3 → avg 88000
dept 2, 95000
...
```

3.4 The GROUP BY Rule

Every column in `SELECT` must be either **in the GROUP BY** or **inside an aggregate**. This is illegal:

```
-- WRONG: name isn't grouped or aggregated
SELECT dept_id, name, AVG(salary) FROM employees GROUP BY dept_id;
```

Because a department has many names — which one would it show? Fix: group by it, or aggregate it, or remove it.

3.5 Grouping by Multiple Columns

```
-- Orders and revenue per customer per month
SELECT customer_id,
       EXTRACT(MONTH FROM order_date) AS mth,
       COUNT(*)      AS orders,
       SUM(amount)   AS revenue
FROM orders
GROUP BY customer_id, EXTRACT(MONTH FROM order_date);
```

❄️ Snowflake: `MONTH(order_date)` is cleaner than `EXTRACT`.

3.6 HAVING — Filtering Groups

`WHERE` filters **rows**; `HAVING` filters **groups** (after aggregation). You can't use an aggregate in `WHERE`.

```
-- Departments with more than 1 employee
SELECT dept_id, COUNT(*) AS headcount
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 1;
```

```
-- Customers who spent over 50,000
SELECT customer_id, SUM(amount) AS total
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > 50000;
```

3.7 WHERE + GROUP BY + HAVING Together

They run in this order: filter rows → group → filter groups.

```
-- Among 2023 orders, customers with 2+ orders
SELECT customer_id, COUNT(*) AS orders_2023
FROM orders
WHERE order_date >= '2023-01-01'      -- filter rows first
GROUP BY customer_id                  -- then group
HAVING COUNT(*) >= 2;                  -- then filter groups
```

3.8 Conditional Aggregation (a power move)

Count/sum rows meeting a condition *within* each group using `CASE`:

```
SELECT dept_id,
       COUNT(*) AS total,
       SUM(CASE WHEN salary >= 100000 THEN 1 ELSE 0 END) AS high_earners
FROM employees
GROUP BY dept_id;
```

❄️ Snowflake shortcut: `COUNT_IF(salary >= 100000)`.

3.9 Rounding & Readability

```
SELECT dept_id, ROUND(AVG(salary), 0) AS avg_salary
FROM employees GROUP BY dept_id;
```

3.10 The Full Logical Order (now complete)

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Memorise this — it explains almost every "why doesn't my query work?" question.

✅ Key Takeaways

1. Aggregates (`COUNT` , `SUM` , `AVG` , `MIN` , `MAX`) collapse many rows into one value.
2. `COUNT(*)` counts rows; `COUNT(col)` ignores NULLs.
3. `GROUP BY X` gives one row per group — use it whenever you want results "per X".
4. Every selected column must be grouped or aggregated.
5. `WHERE` filters rows (before grouping); `HAVING` filters groups (after).
6. `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` = conditional counting within groups.

🏆 Exercises

1. Count how many employees there are, and how many have an email.
2. Show the average salary per department.
3. Show total revenue per product category (join needed — or use `orders` alone by `product_id`).
4. List departments with an average salary above 90,000 (`HAVING`).
5. Count, per department, how many employees earn 100,000+ (conditional aggregation).
6. Show total order amount per customer, only for customers who spent over 20,000.

Next: [Module 04 — JOINS](#) →

Module 04 — JOINS: Combining Tables

The single most important SQL skill. Real data is split across tables; JOINS put it back together. Interviews live here.

4.1 Why Tables Are Split (Normalization)

We don't store the department name in every employee row — we'd repeat it thousands of times. Instead, `employees` stores a `dept_id` that **points to** the `departments` table. JOINS follow those pointers.

employees			departments	
emp_id	dept_id		dept_id	dept_name
1	1	→	1	Engineering
2	1		2	Sales
3	2	→		

4.2 INNER JOIN — Matching Rows from Both Tables

Returns rows where the join condition matches in **both** tables:

```
SELECT e.name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

- `e` and `d` are **aliases** (shorthand for the tables).
- `ON e.dept_id = d.dept_id` is the **join condition** (which rows pair up).
- Plain `JOIN` = `INNER JOIN`.

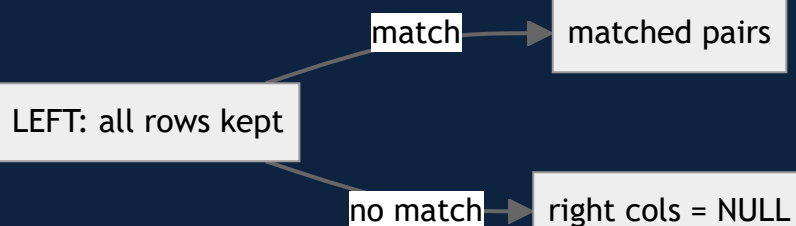
An employee with a `dept_id` that has no match in `departments` is **dropped** by an inner join.

4.3 LEFT JOIN — Keep All Left Rows

Keeps **every** row from the left table; unmatched right-side columns become `NULL`:

```
-- All departments, even those with no employees
SELECT d.dept_name, e.name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id;
```

Use `LEFT JOIN` when you must not lose rows from the main table.



4.4 The Join Types

Join	Keeps
INNER JOIN	only matching rows (both sides)
LEFT JOIN	all left rows + matches
RIGHT JOIN	all right rows + matches
FULL OUTER JOIN	all rows from both sides
CROSS JOIN	every combination (Cartesian product)

`RIGHT JOIN` is just a `LEFT JOIN` written backwards — most people always use `LEFT`.

4.5 The Anti-Join — "Rows With No Match"

A super-common interview task: find rows in A with **no** match in B. Do a `LEFT JOIN` and keep the NULLs:

```
-- Customers who never placed an order
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;      -- no matching order → NULL
```

4.6 Joining Three or More Tables

Chain joins — each `JOIN ... ON` links one more table:

```
SELECT o.order_id, c.name AS customer, p.product_name, o.amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN products p ON o.product_id = p.product_id;
```

4.7 Self Join — a Table Joined to Itself

When rows relate to other rows in the *same* table (e.g., employee → manager):

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id;  -- m = the manager row
```

Treat the two aliases (`e`, `m`) as if they were two separate tables.

4.8 JOIN + GROUP BY — the Everyday Combo

Most analytics = join tables, then aggregate:

```
-- Revenue per product category
SELECT p.category, SUM(o.amount) AS revenue
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

4.9 Common JOIN Mistakes

- **Forgetting the ON clause** → accidental CROSS JOIN (every combination — huge, wrong results).
- **Wrong join type** → an INNER JOIN silently drops rows you needed (use LEFT).
- **Counting after a join** → `COUNT(*)` may double-count if one row matches many; use `COUNT(DISTINCT ...)`.
- **Ambiguous columns** → if both tables have `dept_id`, qualify it: `e.dept_id`.

4.10 A Mental Model

Think of a JOIN as: "for each row on the left, find the matching row(s) on the right and stick their columns on." INNER drops non-matches; LEFT keeps them with NULLs.

✓ Key Takeaways

1. Tables are split to avoid repetition; **JOINS recombine them** via key columns.
2. `INNER JOIN` keeps only matches; `LEFT JOIN` keeps all left rows (+NULLs for non-matches).
3. **Anti-join** = `LEFT JOIN ... WHERE right.key IS NULL` → "rows with no match".
4. Chain joins for 3+ tables; use a **self join** for within-table relationships (employee↔manager).
5. **JOIN + GROUP BY** powers most analytics.
6. Qualify columns (`e.dept_id`), pick the right join type, and beware double-counting.

🏋️ Exercises

1. List each employee with their department name (INNER JOIN).
2. List all departments and their employees, including empty departments (LEFT JOIN).
3. Find customers who never placed an order (anti-join).
4. Show each order with the customer name and product name (3-table join).
5. Show each employee alongside their manager's name (self join).
6. Compute total revenue per product category (join + group by).

Next: [Module 05 — Subqueries & CTEs](#) →

Module 05 — Subqueries & CTEs

Queries inside queries. Once you can nest and name sub-results, you can express almost any question.

5.1 What Is a Subquery?

A **subquery** is a `SELECT` inside another query. The inner query runs first; the outer query uses its result.

```
-- Employees earning above the company average
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Here `(SELECT AVG(salary) FROM employees)` returns one number; the outer query compares each salary to it.

5.2 Three Shapes of Subquery

1) **Scalar** — returns a single value; use it like a value:

```
WHERE salary > (SELECT AVG(salary) FROM employees)
```

2) **List** — returns a column of values; use with `IN` :

```
SELECT name FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Bangalore');
```

3) **Table** — returns rows/columns; use it in `FROM` (a "derived table"):

```
SELECT dept_id, avg_sal
FROM (SELECT dept_id, AVG(salary) AS avg_sal FROM employees GROUP BY dept_id) t
WHERE avg_sal > 90000;
```

5.3 Correlated Subqueries

A **correlated** subquery references the outer row — it re-runs for each outer row:

```
-- Employees earning above THEIR department's average
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees
                  WHERE dept_id = e.dept_id); -- references e.dept_id
```

Powerful, but can be slow (runs per row) — a window function often does this better (Module 06).

5.4 EXISTS / NOT EXISTS

Tests whether a subquery returns **any** rows. Great for "has / has not" questions and safe with NULLs:


```
-- Customers who HAVE ordered
SELECT c.name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

-- Customers who have NOT ordered
SELECT c.name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

`SELECT 1` is a convention — `EXISTS` only cares *whether* rows exist, not what's in them.

5.5 NOT IN vs NOT EXISTS (a NULL trap)

```
-- ⚠️ If the subquery returns any NULL, NOT IN returns NO rows (surprising!)
WHERE customer_id NOT IN (SELECT customer_id FROM orders) -- risky if NULLs
-- ✅ NOT EXISTS handles NULLs correctly
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
```

Prefer `NOT EXISTS` for anti-joins when NULLs are possible.

5.6 CTEs — Common Table Expressions (WITH)

A **CTE** is a named, temporary result you define up front with `WITH`, then use below. It makes complex queries **readable**:

```
WITH dept_avg AS (
  SELECT dept_id, AVG(salary) AS avg_sal
  FROM employees
  GROUP BY dept_id
)
SELECT e.name, e.salary, d.avg_sal
FROM employees e
JOIN dept_avg d ON e.dept_id = d.dept_id
WHERE e.salary > d.avg_sal;
```

Read it top-to-bottom: "first compute `dept_avg`, then use it." Much clearer than a nested subquery.

5.7 Multiple & Chained CTEs

You can define several, and later ones can use earlier ones:

```
WITH
spend AS (
  SELECT customer_id, SUM(amount) AS total
  FROM orders GROUP BY customer_id
),
ranked AS (
  SELECT customer_id, total,
         total > (SELECT AVG(total) FROM spend) AS above_avg
  FROM spend
)
SELECT * FROM ranked WHERE above_avg;
```

5.8 CTE vs Subquery — Which?

- **CTE** — when the logic is complex, reused, or you want readability. Default choice for anything non-trivial.
- **Subquery** — for quick, one-off inline values. They're often interchangeable; CTEs just read better.

5.9 Building Up Complex Queries

The pro workflow: solve it in **layers**. Each CTE is one clear step (clean → aggregate → rank → filter). Build and test one layer at a time. This is how you tackle hard interview questions calmly.

✓ Key Takeaways

1. A **subquery** is a SELECT inside another query — scalar (a value), list (with `IN`), or table (in `FROM`).
2. **Correlated** subqueries reference the outer row and run per-row (often replaceable by window functions).
3. `EXISTS` / `NOT EXISTS` test for the presence of rows — the safe way to do anti-joins.
4. Prefer `NOT EXISTS` over `NOT IN` when NULLs are possible.
5. **CTEs** (`WITH`) name intermediate results for readability; chain several to build complex logic in layers.
6. Solve hard queries **one layer at a time**.

🏋️ Exercises

1. Find employees earning more than the company average (scalar subquery).
2. Find employees in departments located in 'Bangalore' (list subquery with `IN`).
3. Find employees earning above their own department's average (correlated subquery).
4. Find customers who have never ordered, using `NOT EXISTS`.
5. Rewrite exercise 3 using a CTE that computes department averages.
6. Using two chained CTEs, list customers who spent above the average customer spend.

Next: [Module 06 — Window Functions](#) →

Module 06 — Window Functions

The skill that separates intermediate from advanced. Window functions answer "rank", "running total", "compare to previous row" — the hardest interview questions.

6.1 The Big Idea

A **window function** computes a value across a set of rows *related to the current row*, **without collapsing** them. Unlike `GROUP BY` (which reduces many rows to one), a window function **keeps every row** and adds a computed column.

```
GROUP BY: 7 rows → 3 rows (one per group)
WINDOW:   7 rows → 7 rows (+ a new column with the group value on each)
```

The syntax:

```
FUNCTION(...) OVER (PARTITION BY ... ORDER BY ...)
```

- `PARTITION BY` — split rows into groups (like `GROUP BY`, but rows stay).
- `ORDER BY` — order rows within each partition (needed for ranking/running totals).

6.2 Ranking Functions

```
SELECT name, dept_id, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
       RANK() OVER (ORDER BY salary DESC) AS rank,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank
FROM employees;
```

The difference (on ties): | | ROW_NUMBER | RANK | DENSE_RANK | |--|--|--| | values | always unique | gaps after ties | no gaps | |
example | 1,2,3,4 | 1,2,2,4 | 1,2,2,3 |

6.3 PARTITION BY — Restart Per Group

```
-- Rank salaries WITHIN each department
SELECT name, dept_id, salary,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank
FROM employees;
```

The ranking restarts at 1 for each department. This `PARTITION BY` is the key to all "per group" analytics.

6.4 Top-N Per Group (the #1 interview pattern)

Wrap the window in a subquery/CTE and filter on the rank:

```
-- Top 2 earners per department
SELECT dept_id, name, salary FROM (
  SELECT dept_id, name, salary,
         DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
  FROM employees
) t
WHERE rnk <= 2;
```

❄️ Snowflake makes this one line with `QUALIFY` :

```
SELECT dept_id, name, salary FROM employees
QUALIFY DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) <= 2;
```

6.5 Aggregate Windows — Compare Each Row to Its Group

Aggregates can be windows too — every row keeps its value AND sees the group aggregate:

```
SELECT name, dept_id, salary,
       AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY dept_id) AS diff_from_avg
FROM employees;
```

This is impossible with plain `GROUP BY` (which would collapse the rows). Hugely useful.

6.6 Running Totals

Add `ORDER BY` inside `OVER` to accumulate:

```
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
       AS running_total
FROM orders;
```

The **frame** `UNBOUNDED PRECEDING → CURRENT ROW` means "all rows up to and including this one". Partition it for per-group running totals:

```
SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date)
```

6.7 LAG & LEAD — Look at Other Rows

`LAG` = previous row's value, `LEAD` = next row's. Perfect for period-over-period comparisons:

```
-- Month-over-month revenue change
WITH monthly AS (
  SELECT DATE_TRUNC('month', order_date) AS mth, SUM(amount) AS rev
  FROM orders GROUP BY 1
)
SELECT mth, rev,
       LAG(rev) OVER (ORDER BY mth)      AS prev_month,
       rev - LAG(rev) OVER (ORDER BY mth) AS change
FROM monthly;
```

6.8 FIRST_VALUE / LAST_VALUE / NTILE

```
-- Bucket employees into 4 salary quartiles
SELECT name, salary, NTILE(4) OVER (ORDER BY salary DESC) AS quartile
FROM employees;
```

`FIRST_VALUE` / `LAST_VALUE` grab the first/last value in the window (mind the frame for `LAST_VALUE` — see the practice bank #87).

6.9 When to Use Windows vs GROUP BY

- Need **one row per group** (a summary table)? → `GROUP BY` .
 - Need to **keep all rows** and add a comparison/rank/running value? → **window function**.
 - "Rank", "top N per group", "running total", "vs previous/average" → **window function**, every time.
-

✓ Key Takeaways

1. Window functions compute across related rows **without collapsing** them: `FUNC() OVER (PARTITION BY ... ORDER BY ...)` .
2. `ROW_NUMBER` (unique), `RANK` (gaps), `DENSE_RANK` (no gaps) for ranking.
3. `PARTITION BY` restarts the calculation per group — the key to per-group analytics.
4. **Top-N-per-group** = rank in a subquery/CTE, filter the rank (or Snowflake `QUALIFY`).
5. Aggregate windows compare each row to its group average/total; add `ORDER BY` for running totals.
6. `LAG` / `LEAD` compare to previous/next rows (period-over-period).

🏆 Exercises

1. Rank all employees by salary using `RANK` and `DENSE_RANK` ; note the difference on ties.
2. Rank employees by salary **within each department**.
3. Return the top 2 earners per department.
4. Show each employee's salary next to their department average and the difference.
5. Compute a running total of `orders.amount` ordered by date.
6. Show month-over-month revenue change using `LAG` .

Next: [Module 07 — Advanced SQL](#) →

Module 07 — Advanced SQL

Pivots, set operations, recursion, dates, and performance — the toolkit that handles any real-world request.

7.1 CASE — Conditional Logic

SQL's if/else. Use it for labels, bucketing, and conditional aggregation:

```
SELECT name, salary,
       CASE WHEN salary >= 120000 THEN 'High'
            WHEN salary >= 80000  THEN 'Medium'
            ELSE 'Low' END AS band
FROM employees;
```

Conditions are checked top-down; the first TRUE wins. Add `ELSE` for the fallback (else you get NULL).

7.2 Pivoting — Rows into Columns

Turn category rows into columns with conditional aggregation:

```
-- Revenue per category as columns
SELECT
  SUM(CASE WHEN category = 'Electronics' THEN amount ELSE 0 END) AS electronics,
  SUM(CASE WHEN category = 'Furniture'   THEN amount ELSE 0 END) AS furniture
FROM orders o JOIN products p ON o.product_id = p.product_id;
```

❄️ Snowflake has a native `PIVOT` clause; the `CASE` method works everywhere.

7.3 Set Operations — Stacking Results

Combine the results of two queries (same columns):

```
SELECT name FROM employees
UNION
      -- removes duplicates
SELECT name FROM customers;

SELECT name FROM employees
UNION ALL
      -- keeps duplicates (faster)
SELECT name FROM customers;
```

- `UNION` / `UNION ALL` — stack rows.
- `INTERSECT` — rows in both.
- `EXCEPT` (or `MINUS`) — rows in the first but not the second.

7.4 Subtotals — ROLLUP / CUBE / GROUPING SETS

Get group subtotals AND a grand total in one query:

```
SELECT dept_id, COUNT(*) AS headcount
FROM employees
GROUP BY ROLLUP(dept_id); -- rows per dept + a grand-total row (dept_id NULL)
```

`CUBE` gives all combinations; `GROUPING SETS` lets you pick exactly which subtotals.

7.5 Date & Time Functions

The bread and butter of analytics:

```
-- Standard SQL
EXTRACT(YEAR FROM order_date)      -- pull a part
order_date + INTERVAL '7 days'    -- date math (Postgres)

-- Snowflake (cleaner)
YEAR(order_date), MONTH(order_date)
DATEADD('day', 7, order_date)
DATEDIFF('day', start_date, end_date)
DATE_TRUNC('month', order_date)    -- floor to the month
```

Truncating to month/week is the key trick for time-series grouping.

7.6 String Functions

```
UPPER(name), LOWER(name), LENGTH(name)
TRIM(name)                                -- strip whitespace
SUBSTRING(email, 1, 5)                   -- extract part
REPLACE(phone, '-', '')                  -- swap text

-- Snowflake gem:
SPLIT_PART(email, '@', 2)                -- domain from email
LISTAGG(name, ',')                       -- concatenate a group into one string
```

7.7 Recursive CTEs — Hierarchies & Series

For trees (org charts, categories) and generated sequences:

```
-- Full management chain from the CEO down
WITH RECURSIVE org AS (
  SELECT emp_id, name, manager_id, 1 AS level
  FROM employees WHERE manager_id IS NULL      -- anchor: the top
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id, o.level + 1
  FROM employees e JOIN org o ON e.manager_id = o.emp_id -- recursive step
)
SELECT level, name FROM org ORDER BY level;
```

Recursion = an **anchor** (starting rows) + a **recursive member** (rows built from the previous step) until nothing new is produced.

7.8 NULL Handling Functions

```
COALESCE(email, 'no-email')             -- first non-NULL argument
NULLIF(a, b)                             -- NULL if a = b (guards divide-by-zero)

-- Safe division:
revenue / NULLIF(orders, 0)             -- avoids /0 error when orders is 0
```

7.9 Query Performance (practical intuition)

You don't need to be a DBA, but know the basics: - **Filter early** — a selective `WHERE` reduces work downstream. - **Indexes** (in OLTP DBs like Postgres/MySQL) speed up lookups on filtered/joined columns. - **Avoid** `SELECT *` in production — read only what you need. - **Beware functions on filtered columns** — `WHERE YEAR(date)=2023` can prevent index use; prefer `WHERE date >= '2023-01-01' AND date < '2024-01-01'`. - In Snowflake, performance is about **pruning** (clustering) and **warehouse size**, not indexes. - Read the **query plan** (`EXPLAIN`) to see what's slow.

7.10 Putting It Together

Advanced SQL is mostly **combining** these tools: a CTE that cleans → a window that ranks → a CASE that labels → a pivot that reshapes. Build in layers, test each step.

✓ Key Takeaways

1. `CASE` = conditional logic; powers labels, bucketing, and pivots (rows→columns).
2. Set ops: `UNION` (/ALL), `INTERSECT`, `EXCEPT` stack/compare result sets.
3. `ROLLUP` / `CUBE` / `GROUPING SETS` add subtotals and grand totals.
4. Master **date functions** (`DATE_TRUNC`, `DATEDIFF`) and **string functions** (`SPLIT_PART`, `LISTAGG`).
5. **Recursive CTEs** handle hierarchies and generated series (anchor + recursive step).
6. `COALESCE` / `NULLIF` handle NULLs and guard divide-by-zero; filter early and avoid functions on filtered columns for speed.

🔧 Exercises

1. Label each employee's salary as High/Medium/Low with `CASE`.
2. Pivot: show total revenue for Electronics vs Furniture as two columns.
3. Use `UNION` to list all employee and customer names in one column.
4. Use `ROLLUP` to show headcount per department plus a grand total.
5. Extract the email domain of each employee (`SPLIT_PART` or `SUBSTRING`).
6. Write a recursive CTE that lists numbers 1 to 10.

Next: [Module 08 — SQL for Interviews & Real Work](#) →

Module 08 — SQL for Interviews & Real Work

You know the syntax. This module makes you *fast, correct, and calm* — in interviews and on the job.

8.1 How to Attack Any SQL Problem

A repeatable method that works even on hard questions:

1. Restate the ask in one sentence. What's ONE row of the output?
2. Identify the tables and the keys that join them.
3. Decide the grain: per what? (per customer / per day / per order)
4. Build in layers (CTEs): filter → join → aggregate → rank → label.
5. Test each layer's output before adding the next.
6. Sanity-check: row counts, a known example, NULLs.

Interviewers care about your **approach** as much as the answer. Narrate it.

8.2 The Patterns That Cover 90% of Interviews

Pattern	Tool
Nth highest value	<code>DENSE_RANK</code> / <code>LIMIT OFFSET</code>
Top-N per group	window rank + filter (<code>QUALIFY</code>)
Rows with no match	<code>LEFT JOIN ... IS NULL</code> / <code>NOT EXISTS</code>
Duplicates	<code>GROUP BY ... HAVING COUNT(*)>1</code> or window count
Running total	<code>SUM()</code> OVER (<code>ORDER BY ...</code>)
Period-over-period	<code>LAG</code> / <code>LEAD</code>
Consecutive streaks	gaps-and-islands (row_number trick)
Pivot	conditional aggregation / <code>PIVOT</code>
Hierarchy	recursive CTE

Master these from the [100-problem bank](#) and you'll recognise almost any question instantly.

8.3 Classic Questions (know these cold)

Second-highest salary:

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Duplicate emails:

```
SELECT email FROM employees GROUP BY email HAVING COUNT(*) > 1;
```

Employees earning more than their manager:

```
SELECT e.name FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Customers who never ordered:

```
SELECT c.name FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

8.4 Writing Readable SQL (real-world discipline)

- **Uppercase keywords**, lowercase identifiers.
- One clause per line; indent sub-parts.
- **Meaningful aliases** (`e` for employees, `o` for orders).
- **CTEs over deep nesting** — name each step.
- Comment the *why*, not the obvious *what*.
- Consistent formatting — your future self and teammates thank you.

8.5 Correctness Habits That Prevent Bugs

- After a join, ask: *could this duplicate rows?* If yes, `COUNT(DISTINCT ...)`.
- With NULLs, prefer `NOT EXISTS` over `NOT IN`.
- Integer division truncates — use `100.0 * a / b` for percentages.
- Guard divide-by-zero with `NULLIF(denominator, 0)`.
- `HAVING` for aggregate filters, `WHERE` for row filters.
- Always test with an edge case (empty group, ties, a NULL).

8.6 Performance on the Job

- Select only needed columns; filter early.
- Understand your engine: **OLTP** (Postgres/MySQL) uses indexes; **Snowflake** uses pruning + warehouse sizing.
- Read the query plan (`EXPLAIN`) when something is slow.
- Aggregate at the right grain — don't join then dedupe if you can aggregate first.

8.7 SQL Across Engines

Core SQL is portable, but dialects differ: | Task | Standard/Postgres | Snowflake | |-----|-----|-----| | Case-insensitive match | `ILIKE` (PG) | `ILIKE` | | Date part | `EXTRACT(MONTH FROM d)` | `MONTH(d)` | | Date diff | `d2 - d1` | `DATEDIFF('day', d1, d2)` | | String split | `SPLIT_PART` (PG/SF) | `SPLIT_PART` | | Filter window | subquery | `QUALIFY` | | Limit | `LIMIT n` | `LIMIT n / FETCH FIRST n` |

Learn one dialect deeply (we teach standard + Snowflake); translating is then easy.

8.8 Your Practice Plan

1. Work the [100 problems](#): Easy → Medium → Hard.
2. **Try before peeking.** The struggle is the learning.
3. Redo missed problems a week later.
4. Time yourself on Medium/Hard (interview simulation).
5. Explain your solution out loud — that's what interviews test.

✓ Key Takeaways

1. Attack problems with a **method**: restate → identify tables/grain → build in layers → test → sanity-check.
2. Learn the **9 core patterns** — they cover ~90% of interviews.
3. Know the **classic questions** (2nd-highest, duplicates, more-than-manager, never-ordered) cold.
4. Write **readable** SQL (CTEs, aliases, formatting) — it's judged.
5. Build **correctness habits**: dedupe after joins, `NOT EXISTS` for NULLs, decimal math, `NULLIF` guards.
6. Core SQL is portable; know your engine's dialect and performance model.

🏆 Exercises

1. Write the second-highest salary query, then generalise it to Nth-highest.
2. Find all duplicate emails in `employees`.
3. Find employees earning more than their manager.
4. List customers who never ordered (two ways: `LEFT JOIN` and `NOT EXISTS`).
5. For each department, return the top earner (window function).
6. Pick 5 problems from the Hard bank and solve them timed.

🎓 Course Complete!

You now have the full SQL toolkit — from `SELECT` to recursion — plus the interview method. Next: grind the [100-problem bank](#), then apply SQL in the **Snowflake** or **Data Science** courses.

100 SQL Practice Problems — PJ's Academy

LeetCode-style problems. Every solution comes in **standard SQL** AND **Snowflake**, with line-by-line comments explaining the *why*, not just the *what*.

First, run the [shared schema](#) to create the sample tables (employees, departments, customers, products, orders).

The Bank

Tier	Problems	Focus	File
 Easy	1–35	SELECT, WHERE, ORDER BY, basic aggregates, simple joins	01-easy.md
 Medium	36–75	Multi-joins, subqueries, CTEs, GROUP BY + HAVING, CASE	02-medium.md
 Hard	76–100	Window functions, recursion, top-N-per-group, gaps & islands	03-hard.md

How To Practice

1. Read the problem. **Try it yourself first** (cover the solution).
2. Compare with the standard-SQL solution + comments.
3. Study the **Snowflake** version — note where it differs (e.g., `QUALIFY`, date functions).
4. Mark problems you missed. **Redo them in a week.**

SQL vs Snowflake — quick orientation

Most SQL is identical across engines. Snowflake adds power features you'll see used here: - `QUALIFY` — filter window-function results without a subquery (Snowflake gem). - `DATEDIFF`, `DATEADD`, `DATE_TRUNC` — clean date math. - `ILIKE` — case-insensitive matching. - `LISTAGG` — string aggregation. - `::type` — casting shorthand. Where a problem's solution is identical in both, we note "Snowflake: identical" and just add any idiomatic tip.

● Easy — Problems 1–35

Foundations: SELECT, WHERE, ORDER BY, LIMIT, basic aggregates, simple joins. Run [00-schema.sql](#) first. Try each yourself before reading the solution.

1. Select all employees

Task: Return every column for all employees.

```
-- SELECT * returns all columns; FROM names the table
SELECT * FROM employees;
```

❄ **Snowflake:** identical. 💬 The simplest query. * = all columns. In real code, list columns explicitly instead of * for clarity and speed.

2. Names and salaries only

Task: Return just the name and salary of every employee.

```
-- List the specific columns you want, comma-separated
SELECT name, salary FROM employees;
```

❄ **Snowflake:** identical. 💬 Selecting specific columns is faster and clearer than *.

3. Employees earning more than 100000

```
-- WHERE filters rows; only rows where the condition is TRUE are returned
SELECT name, salary
FROM employees
WHERE salary > 100000;
```

❄ **Snowflake:** identical. 💬 WHERE runs before the columns are returned — it decides which rows survive.

4. Employees in department 1

```
SELECT name FROM employees
WHERE dept_id = 1; -- = tests exact equality
```

❄ **Snowflake:** identical.

5. Employees NOT in department 2

```
SELECT name FROM employees
WHERE dept_id <> 2; -- <> means "not equal" (!= also works)
```

❄ **Snowflake:** identical. 💬 ⚠ Rows where dept_id is NULL are **not** returned — NULL comparisons yield UNKNOWN, not TRUE.

6. Sort employees by salary (highest first)

```
SELECT name, salary FROM employees
ORDER BY salary DESC;  -- DESC = descending; default is ASC
```

❄️ **Snowflake:** identical.

7. Top 3 highest-paid employees

```
SELECT name, salary FROM employees
ORDER BY salary DESC
LIMIT 3;  -- LIMIT caps the number of rows returned
```

❄️ **Snowflake:** identical (`LIMIT 3` works; `FETCH FIRST 3 ROWS ONLY` also valid). 💡 Sort first, then limit — you get the top 3 by salary.

8. Count the employees

```
SELECT COUNT(*) AS total_employees
FROM employees;  -- COUNT(*) counts rows; AS renames the output column
```

❄️ **Snowflake:** identical.

9. Number of employees with an email

```
-- COUNT(column) ignores NULLs, so this counts only non-NULL emails
SELECT COUNT(email) AS with_email
FROM employees;
```

❄️ **Snowflake:** identical. 💡 Key distinction: `COUNT(*)` counts all rows; `COUNT(col)` counts non-NULLs. Nina's NULL email is excluded.

10. Highest and lowest salary

```
SELECT MAX(salary) AS highest,
       MIN(salary) AS lowest
FROM employees;
```

❄️ **Snowflake:** identical.

11. Average salary

```
SELECT AVG(salary) AS avg_salary
FROM employees;
```

❄️ **Snowflake:** identical. 💡 `AVG` ignores NULLs. Round it with `ROUND(AVG(salary), 2)`.

12. Total payroll

```
SELECT SUM(salary) AS total_payroll
FROM employees;
```

❄️ **Snowflake:** identical.

13. Employees hired after 2021

```
SELECT name, hire_date FROM employees
WHERE hire_date > '2021-12-31'; -- dates compare like numbers
```

❄️ **Snowflake:** identical. Tip: `WHERE YEAR(hire_date) > 2021` also works in Snowflake.

14. Employees hired in 2021

```
SELECT name FROM employees
WHERE hire_date BETWEEN '2021-01-01' AND '2021-12-31';
-- BETWEEN is inclusive on both ends
```

❄️ **Snowflake:** identical, or `WHERE YEAR(hire_date) = 2021`.

15. Employees whose name starts with 'A'

```
SELECT name FROM employees
WHERE name LIKE 'A%'; -- % = any sequence of characters
```

❄️ **Snowflake:** identical. Case-insensitive version: `WHERE name ILIKE 'a%'`. 💡 `%` = zero or more chars, `_` = exactly one char.

16. Salaries between 80000 and 120000

```
SELECT name, salary FROM employees
WHERE salary BETWEEN 80000 AND 120000;
```

❄️ **Snowflake:** identical.

17. Employees in departments 1 or 3

```
SELECT name, dept_id FROM employees
WHERE dept_id IN (1, 3); -- IN = matches any value in the list
```

❄️ **Snowflake:** identical. 💡 `IN (1,3)` is cleaner than `dept_id = 1 OR dept_id = 3`.

18. Employees with no manager (the CEO)

```
SELECT name FROM employees
WHERE manager_id IS NULL; -- use IS NULL, never = NULL
```

❄️ **Snowflake:** identical. 💡 ⚠️ `= NULL` never works — NULL isn't a value you can equal. Always `IS NULL` / `IS NOT NULL`.

19. Distinct departments that have employees

```
SELECT DISTINCT dept_id FROM employees; -- DISTINCT removes duplicates
```

❄️ **Snowflake:** identical.

20. Rename salary to monthly (aliasing)

```
SELECT name, salary AS monthly_salary
FROM employees;
```

❄️ **Snowflake:** identical.

21. Employees ordered by department, then salary

```
SELECT name, dept_id, salary FROM employees
ORDER BY dept_id ASC, salary DESC; -- sort by dept, then salary within dept
```

❄️ **Snowflake:** identical.

22. All customers from Mumbai

```
SELECT name FROM customers
WHERE city = 'Mumbai';
```

❄️ **Snowflake:** identical (`WHERE city ILIKE 'mumbai'` for case-insensitive).

23. Products cheaper than 10000

```
SELECT product_name, price FROM products
WHERE price < 10000;
```

❄️ **Snowflake:** identical.

24. Count products per category

```
-- GROUP BY collapses rows sharing a category into one group;
-- COUNT(*) then counts rows within each group
SELECT category, COUNT(*) AS num_products
FROM products
GROUP BY category;
```

❄️ **Snowflake:** identical. 💡 Whenever you aggregate "per something", that something goes in `GROUP BY`.

25. Average price per category

```
SELECT category, ROUND(AVG(price), 2) AS avg_price
FROM products
GROUP BY category;
```

❄️ **Snowflake:** identical.

26. Number of employees per department


```
SELECT dept_id, COUNT(*) AS headcount
FROM employees
GROUP BY dept_id;
```

❄ **Snowflake:** identical.

27. Total order amount per customer

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id;
```

❄ **Snowflake:** identical.

28. Departments with more than 1 employee

```
SELECT dept_id, COUNT(*) AS headcount
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 1;  -- HAVING filters GROUPS (WHERE filters rows)
```

❄ **Snowflake:** identical. 💬 `WHERE` filters rows *before* grouping; `HAVING` filters *after* grouping. You can't use aggregates in `WHERE`.

29. List employees with their department name (JOIN)

```
-- JOIN matches each employee to its department via the shared dept_id
SELECT e.name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

❄ **Snowflake:** identical. 💬 `e` and `d` are table aliases — shorthand so you can write `e.name` instead of `employees.name`.

30. Employees and their department location

```
SELECT e.name, d.location
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

❄ **Snowflake:** identical.

31. Orders with the customer's name

```
SELECT o.order_id, c.name, o.amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id;
```

❄ **Snowflake:** identical.

32. Orders with product names

```
SELECT o.order_id, p.product_name, o.quantity
FROM orders o
JOIN products p ON o.product_id = p.product_id;
```

❄️ **Snowflake:** identical.

33. All departments, even those with no employees (LEFT JOIN)

```
-- LEFT JOIN keeps every department; employees with no match show as NULL
SELECT d.dept_name, e.name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id;
```

❄️ **Snowflake:** identical. 💬 A plain `JOIN` (inner) would drop departments with no employees. `LEFT JOIN` keeps the left table's rows regardless.

34. Count orders per product (including unordered products)

```
SELECT p.product_name, COUNT(o.order_id) AS times_ordered
FROM products p
LEFT JOIN orders o ON p.product_id = o.product_id
GROUP BY p.product_name;
```

❄️ **Snowflake:** identical. 💬 `COUNT(o.order_id)` counts non-NULLs, so unordered products correctly show 0 (not 1).

35. Employees earning above the company average

```
-- The subquery computes one number (the average); the outer query compares to it
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

❄️ **Snowflake:** identical. 💬 A **scalar subquery** returns a single value you can compare against — your first taste of subqueries (Medium tier goes deeper).

➡️ Continue to 🟡 **Medium** — Problems 36–75

📖 **SQL Mastery** — *PJ's Academy*

● Medium — Problems 36–75

Multi-table joins, subqueries, CTEs, CASE, conditional aggregation, anti-joins, date/string logic — the bread and butter of SQL interviews. Run `00-schema.sql` first.

36. Employees who earn more than their manager

(Classic interview Q — self join)

```
-- Join employees to themselves: e = worker, m = their manager
SELECT e.name AS employee, e.salary, m.name AS manager
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

❄️ **Snowflake:** identical. 💡 A **self join** treats one table as two. `m.emp_id = e.manager_id` links each worker to their boss.

37. Each employee with their manager's name (show CEO too)

```
-- LEFT JOIN so the CEO (manager_id NULL) still appears
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

❄️ **Snowflake:** identical. 💡 With an inner join the CEO would vanish (no manager to match). `LEFT JOIN` keeps them, manager = NULL.

38. Customers who never placed an order

(Classic — anti join)

```
-- LEFT JOIN then keep only rows with no matching order
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

❄️ **Snowflake:** identical. 💡 The unmatched customers get NULL order columns; filtering `IS NULL` isolates them. Alternative: `WHERE customer_id NOT IN (SELECT customer_id FROM orders)`.

39. Same, using NOT EXISTS

```
SELECT c.name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

❄️ **Snowflake:** identical. 💡 `NOT EXISTS` is often the fastest anti-join and handles NULLs safely (unlike `NOT IN`).

40. Second highest salary

(The most-asked SQL interview question)

```
-- Highest salary that is strictly below the maximum
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

❄️ **Snowflake:** identical. (Window-function version appears in Hard #77.) 💡 Robust even with ties. Returns NULL if there's no second value — usually the desired behaviour.

41. Departments and their total salary

```
SELECT d.dept_name, SUM(e.salary) AS total_salary
FROM departments d
JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

❄️ **Snowflake:** identical.

42. Department with the highest average salary

```
SELECT dept_id, AVG(salary) AS avg_sal
FROM employees
GROUP BY dept_id
ORDER BY avg_sal DESC
LIMIT 1;
```

❄️ **Snowflake:** identical. 💡 Group → average → sort → take one. (Ties broken arbitrarily; window functions handle ties better — Hard tier.)

43. Count employees hired per year

```
-- Extract the year, then group by it
SELECT EXTRACT(YEAR FROM hire_date) AS yr, COUNT(*) AS hires
FROM employees
GROUP BY EXTRACT(YEAR FROM hire_date)
ORDER BY yr;
```

❄️ **Snowflake:** `SELECT YEAR(hire_date) AS yr, COUNT(*) ... GROUP BY YEAR(hire_date)` — `YEAR()` is cleaner.

44. Label salaries as High / Medium / Low (CASE)

```
SELECT name, salary,
CASE
  WHEN salary >= 120000 THEN 'High'
  WHEN salary >= 80000 THEN 'Medium'
  ELSE 'Low'
END AS salary_band
FROM employees;
```

❄️ **Snowflake:** identical. 💡 `CASE` is SQL's if/else. Conditions checked top-down; first TRUE wins.

45. Count employees in each salary band (CASE + GROUP BY)

```
SELECT
  CASE WHEN salary >= 120000 THEN 'High'
        WHEN salary >= 80000 THEN 'Medium'
        ELSE 'Low' END AS band,
  COUNT(*) AS n
FROM employees
GROUP BY
  CASE WHEN salary >= 120000 THEN 'High'
        WHEN salary >= 80000 THEN 'Medium'
        ELSE 'Low' END;
```

❄️ **Snowflake:** identical (can also `GROUP BY band` — Snowflake allows grouping by alias).

46. Conditional aggregation — count high earners per department

```
-- SUM(CASE...) counts rows meeting a condition, per group
SELECT dept_id,
  SUM(CASE WHEN salary >= 100000 THEN 1 ELSE 0 END) AS high_earners,
  COUNT(*) AS total
FROM employees
GROUP BY dept_id;
```

❄️ **Snowflake:** identical (or `COUNT_IF(salary >= 100000)` — a Snowflake shortcut). 🗨️ `SUM(CASE WHEN cond THEN 1 ELSE 0 END)` is the universal "count rows matching a condition within a group" pattern.

47. Total revenue per product category

```
SELECT p.category, SUM(o.amount) AS revenue
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

❄️ **Snowflake:** identical.

48. Customers and their total spend, highest first

```
SELECT c.name, SUM(o.amount) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name
ORDER BY total_spent DESC;
```

❄️ **Snowflake:** identical.

49. Customers who spent more than 50000

```
SELECT c.name, SUM(o.amount) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name
HAVING SUM(o.amount) > 50000;
```

❄️ **Snowflake:** identical. 💡 Filter on an aggregate → `HAVING`, not `WHERE`.

50. Average order value per customer using a CTE

```
-- A CTE (WITH) is a named temporary result you can reference below
WITH customer_orders AS (
  SELECT customer_id, COUNT(*) AS num_orders, SUM(amount) AS total
  FROM orders
  GROUP BY customer_id
)
SELECT customer_id, total / num_orders AS avg_order_value
FROM customer_orders;
```

❄️ **Snowflake:** identical. 💡 CTEs make queries readable — build the summary once, use it below.

51. Products never ordered

```
SELECT product_name FROM products
WHERE product_id NOT IN (SELECT product_id FROM orders);
```

❄️ **Snowflake:** identical. 💡 ⚠️ If the subquery could contain NULLs, `NOT IN` breaks — prefer `NOT EXISTS` there. Here `product_id` is never NULL, so it's safe.

52. Each department's employee count and average salary

```
SELECT d.dept_name,
       COUNT(e.emp_id) AS headcount,
       ROUND(AVG(e.salary), 0) AS avg_salary
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

❄️ **Snowflake:** identical. 💡 `LEFT JOIN` + `COUNT(e.emp_id)` → empty departments correctly show 0.

53. Orders above the average order amount (correlated idea)

```
SELECT order_id, amount FROM orders
WHERE amount > (SELECT AVG(amount) FROM orders);
```

❄️ **Snowflake:** identical.

54. Each customer's most recent order date

```
SELECT customer_id, MAX(order_date) AS last_order
FROM orders
GROUP BY customer_id;
```

❄️ **Snowflake:** identical.

55. Number of distinct products each customer bought

```
SELECT customer_id, COUNT(DISTINCT product_id) AS unique_products
FROM orders
GROUP BY customer_id;
```

❄️ **Snowflake:** identical. 🗨️ `COUNT(DISTINCT ...)` counts unique values — a customer buying the same product twice counts once.

56. Combine employee and customer names into one list (UNION)

```
-- UNION stacks two result sets and removes duplicates
SELECT name, 'employee' AS type FROM employees
UNION
SELECT name, 'customer' AS type FROM customers;
```

❄️ **Snowflake:** identical. Use `UNION ALL` to keep duplicates (faster — no dedupe).

57. Replace NULL email with a placeholder (COALESCE)

```
-- COALESCE returns the first non-NULL argument
SELECT name, COALESCE(email, 'no-email@pjs.com') AS email
FROM employees;
```

❄️ **Snowflake:** identical (`IFNULL` and `NVL` also work).

58. Employees whose name contains 'a' (case-insensitive)

```
SELECT name FROM employees
WHERE LOWER(name) LIKE '%a%';
```

❄️ **Snowflake:** `WHERE name ILIKE '%a%'` — `ILIKE` is built-in case-insensitive.

59. Length of each employee's name

```
SELECT name, LENGTH(name) AS name_length
FROM employees;
```

❄️ **Snowflake:** identical (`LENGTH` works; `LEN` also).

60. Uppercase department names

```
SELECT UPPER(dept_name) AS dept_upper FROM departments;
```

❄️ **Snowflake:** identical.

61. Extract domain from email

```
-- SUBSTRING from the char after '@' to the end
SELECT name,
       SUBSTRING(email FROM POSITION('@' IN email) + 1) AS domain
FROM employees
WHERE email IS NOT NULL;
```

❄️ **Snowflake:** `SPLIT_PART(email, '@', 2) AS domain` — far cleaner. 💡 Learn Snowflake's `SPLIT_PART` — it beats manual substring math.

62. Months an employee has been at the company

```
SELECT name,
       (EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM hire_date)) * 12
       + (EXTRACT(MONTH FROM CURRENT_DATE) - EXTRACT(MONTH FROM hire_date)) AS months
FROM employees;
```

❄️ **Snowflake:** `DATEDIFF('month', hire_date, CURRENT_DATE) AS months` — one clean function. 💡 Snowflake's `DATEDIFF(part, start, end)` is a huge quality-of-life win over manual date math.

63. Orders placed in June 2023

```
SELECT order_id, order_date FROM orders
WHERE order_date BETWEEN '2023-06-01' AND '2023-06-30';
```

❄️ **Snowflake:** identical, or `WHERE DATE_TRUNC('month', order_date) = '2023-06-01'`.

64. Revenue per month

```
SELECT EXTRACT(MONTH FROM order_date) AS mth, SUM(amount) AS revenue
FROM orders
GROUP BY EXTRACT(MONTH FROM order_date)
ORDER BY mth;
```

❄️ **Snowflake:** `SELECT DATE_TRUNC('month', order_date) AS mth, SUM(amount) ... GROUP BY 1` — `DATE_TRUNC` keeps the year too.

65. Departments where every employee earns > 60000 (NOT EXISTS)

```
SELECT d.dept_name
FROM departments d
WHERE NOT EXISTS (
  SELECT 1 FROM employees e
  WHERE e.dept_id = d.dept_id AND e.salary <= 60000
)
AND EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
```

❄️ **Snowflake:** identical. 💡 "All employees satisfy X" = "no employee violates X". The extra `EXISTS` excludes empty departments.

66. Top 2 highest-paid employees per... (preview — full version in Hard)


```
-- Simple version: overall top 2 (per-group needs window functions, Hard #82)
SELECT name, salary FROM employees
ORDER BY salary DESC
LIMIT 2;
```

❄️ **Snowflake:** identical.

67. Customers with more than one order

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

❄️ **Snowflake:** identical.

68. Average quantity per product category

```
SELECT p.category, AVG(o.quantity) AS avg_qty
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category;
```

❄️ **Snowflake:** identical.

69. Percentage of total revenue by category

```
SELECT p.category,
       SUM(o.amount) AS revenue,
       ROUND(100.0 * SUM(o.amount) / (SELECT SUM(amount) FROM orders), 1) AS pct
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category;
```

❄️ **Snowflake:** identical. (Window `RATIO_TO_REPORT` does this elegantly — Hard #88.) 💡 `100.0 *` (not `100 *`) forces decimal division — a common gotcha with integer math.

70. Employees in the same department as 'Ravi'

```
SELECT name FROM employees
WHERE dept_id = (SELECT dept_id FROM employees WHERE name = 'Ravi')
AND name <> 'Ravi';
```

❄️ **Snowflake:** identical.

71. Manager and how many people they manage

```
SELECT m.name AS manager, COUNT(e.emp_id) AS reports
FROM employees m
JOIN employees e ON e.manager_id = m.emp_id
GROUP BY m.name
ORDER BY reports DESC;
```

❄️ **Snowflake:** identical.

72. Three-table join: customer, product, amount per order

```
SELECT o.order_id, c.name AS customer, p.product_name, o.amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN products p ON o.product_id = p.product_id;
```

❄️ **Snowflake:** identical. 💡 Chain joins — each `JOIN ... ON` adds one more table on a matching key.

73. Categories with revenue above the average category revenue (CTE)

```
WITH cat_rev AS (
  SELECT p.category, SUM(o.amount) AS revenue
  FROM orders o JOIN products p ON o.product_id = p.product_id
  GROUP BY p.category
)
SELECT category, revenue FROM cat_rev
WHERE revenue > (SELECT AVG(revenue) FROM cat_rev);
```

❄️ **Snowflake:** identical.

74. First and last order date overall, and the span

```
SELECT MIN(order_date) AS first_order,
       MAX(order_date) AS last_order
FROM orders;
```

❄️ **Snowflake:** add `DATEDIFF('day', MIN(order_date), MAX(order_date)) AS span_days`.

75. Flag whether each employee is above or below company average (CASE + subquery)

```
SELECT name, salary,
       CASE WHEN salary > (SELECT AVG(salary) FROM employees)
         THEN 'Above average' ELSE 'Below average' END AS vs_avg
FROM employees;
```

❄️ **Snowflake:** identical.

➔ Continue to 🔴 Hard — Problems 76–100

📖 SQL Mastery — PJ's Academy

● Hard — Problems 76–100

The interview-deciding tier: window functions, ranking, running totals, LAG/LEAD, top-N-per-group, gaps & islands, pivots, and recursion. This is where Snowflake's `QUALIFY` shines. Run `00-schema.sql` first.

A few problems introduce a tiny extra table (given inline) for patterns the base schema can't show (e.g., logins for streaks).

76. Rank employees by salary (RANK)

```
-- A window function computes across a set of rows "related to" the current row.
-- RANK() OVER (ORDER BY salary DESC) numbers rows by salary; ties share a rank.
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

❄️ **Snowflake:** identical. 💬 `RANK` leaves gaps after ties (1,2,2,4); `DENSE_RANK` doesn't (1,2,2,3); `ROW_NUMBER` is always unique (1,2,3,4).

77. Second highest salary (window-function way)

```
SELECT DISTINCT salary AS second_highest
FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) t
WHERE rnk = 2;
```

❄️ **Snowflake (cleaner with QUALIFY):**

```
SELECT DISTINCT salary
FROM employees
QUALIFY DENSE_RANK() OVER (ORDER BY salary DESC) = 2;
```

💬 `DENSE_RANK = 2` gives the 2nd distinct salary. `QUALIFY` filters window results without a subquery — a Snowflake superpower.

78. Nth highest salary (generalise to N=3)

```
SELECT DISTINCT salary
FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) t
WHERE rnk = 3;
```

❄️ **Snowflake:** `... QUALIFY DENSE_RANK() OVER (ORDER BY salary DESC) = 3;` 💬 Change the number to get any Nth. Handles ties correctly via `DENSE_RANK`.

79. Highest-paid employee in each department (top-1-per-group)

```
SELECT dept_id, name, salary
FROM (
  SELECT dept_id, name, salary,
         ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
  FROM employees
) t
WHERE rn = 1;
```

❄️ **Snowflake:**

```
SELECT dept_id, name, salary
FROM employees
QUALIFY ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) = 1;
```

💡 `PARTITION BY dept_id` restarts the numbering per department — the key idea behind "per group" analytics.

80. Salary rank within each department

```
SELECT name, dept_id, salary,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank
FROM employees;
```

❄️ **Snowflake:** identical.

81. Each employee's salary vs their department average

```
-- AVG as a window keeps every row AND shows the group average alongside
SELECT name, dept_id, salary,
       ROUND(AVG(salary) OVER (PARTITION BY dept_id), 0) AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY dept_id) AS diff
FROM employees;
```

❄️ **Snowflake:** identical. 💡 Unlike `GROUP BY` (which collapses rows), a window aggregate **keeps every row** and attaches the group value — perfect for comparisons.

82. Top 2 earners per department

```
SELECT dept_id, name, salary
FROM (
  SELECT dept_id, name, salary,
         DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
  FROM employees
) t
WHERE rnk <= 2;
```

❄️ **Snowflake:** `... QUALIFY DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) <= 2;` 💡 The canonical "top-N-per-group" — memorise this shape; it appears in nearly every SQL interview.

83. Running total of revenue by order date

```
-- SUM as a window with ORDER BY accumulates row by row
SELECT order_id, order_date, amount,
       SUM(amount) OVER (ORDER BY order_date, order_id
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
       AS running_total
FROM orders;
```

❄️ **Snowflake:** identical. 💬 The frame `UNBOUNDED PRECEDING → CURRENT ROW` = "everything up to and including this row" = a running total.

84. Running total of spend per customer

```
SELECT customer_id, order_date, amount,
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS cust_running
FROM orders;
```

❄️ **Snowflake:** identical. 💬 `PARTITION BY customer_id` resets the accumulation for each customer.

85. Month-over-month revenue change (LAG)

```
WITH monthly AS (
  SELECT DATE_TRUNC('month', order_date) AS mth, SUM(amount) AS revenue
  FROM orders GROUP BY 1
)
SELECT mth, revenue,
       LAG(revenue) OVER (ORDER BY mth) AS prev_month,
       revenue - LAG(revenue) OVER (ORDER BY mth) AS change
FROM monthly;
```

❄️ **Snowflake:** identical (`DATE_TRUNC` is native). 💬 `LAG` grabs the previous row's value — the standard tool for period-over-period comparisons.

86. Days between each customer's consecutive orders (LAG on dates)

```
SELECT customer_id, order_date,
       LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_order,
       DATEDIFF('day',
               LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date),
               order_date) AS days_since_prev
FROM orders;
```

❄️ **Snowflake:** identical. (Standard SQL: `order_date - LAG(order_date) OVER (...)` in Postgres.)

87. First and latest order per customer in one row (FIRST_VALUE / LAST_VALUE)

```
SELECT DISTINCT customer_id,
       FIRST_VALUE(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS first_order,
       LAST_VALUE(order_date) OVER (PARTITION BY customer_id ORDER BY order_date
                                   ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_order
FROM orders;
```

❄️ **Snowflake:** identical. 💬 ⚠️ `LAST_VALUE` needs the full-frame clause, else it only sees up to the current row (a classic bug).

88. Each category's share of total revenue (RATIO_TO_REPORT)

```
SELECT p.category,
       SUM(o.amount) AS revenue,
       ROUND(RATIO_TO_REPORT(SUM(o.amount)) OVER () * 100, 1) AS pct_of_total
FROM orders o JOIN products p ON o.product_id = p.product_id
GROUP BY p.category;
```

❄️ **Snowflake:** identical (`RATIO_TO_REPORT` is native). 💡 Standard-SQL fallback: `100.0 * SUM(...) / SUM(SUM(...)) OVER ()`.

89. Quartile (NTILE) of employees by salary

```
-- NTILE(4) splits rows into 4 roughly equal buckets
SELECT name, salary,
       NTILE(4) OVER (ORDER BY salary DESC) AS salary_quartile
FROM employees;
```

❄️ **Snowflake:** identical.

90. Cumulative % of employees (running count / total)

```
SELECT name, salary,
       ROUND(100.0 * ROW_NUMBER() OVER (ORDER BY salary DESC)
            / COUNT(*) OVER (), 1) AS cumulative_pct
FROM employees;
```

❄️ **Snowflake:** identical. 💡 `COUNT(*) OVER ()` (empty OVER) = grand total on every row — handy denominator.

91. Median salary

```
-- MEDIAN is a built-in ordered aggregate
SELECT MEDIAN(salary) AS median_salary FROM employees;
```

❄️ **Snowflake:** identical (`MEDIAN` native; also `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary)`). 💡 Standard SQL (Postgres): `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary)`.

92. Pivot: revenue per category as columns (conditional aggregation)

```
SELECT
  SUM(CASE WHEN p.category = 'Electronics' THEN o.amount ELSE 0 END) AS electronics,
  SUM(CASE WHEN p.category = 'Furniture' THEN o.amount ELSE 0 END) AS furniture
FROM orders o JOIN products p ON o.product_id = p.product_id;
```

❄️ **Snowflake (native PIVOT):**

```
SELECT * FROM (
  SELECT p.category, o.amount FROM orders o JOIN products p ON o.product_id=p.product_id
) PIVOT (SUM(amount) FOR category IN ('Electronics', 'Furniture'));
```

💡 Conditional aggregation is the portable pivot; Snowflake's `PIVOT` is the native shortcut.

93. Consecutive login days per user — the "islands" problem

(Add a small table for this classic)

```
-- Logins(user_id, login_date). Find streaks of consecutive days.
-- Trick: (date - ROW_NUMBER days) is constant within a consecutive run
WITH ranked AS (
  SELECT user_id, login_date,
         DATEADD('day',
                 -ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date),
                 login_date) AS grp
  FROM logins
)
SELECT user_id, MIN(login_date) AS streak_start,
       MAX(login_date) AS streak_end, COUNT(*) AS streak_len
FROM ranked
GROUP BY user_id, grp
ORDER BY user_id, streak_start;
```

❄️ **Snowflake:** identical (`DATEADD` native). Postgres: `login_date - ROW_NUMBER() OVER(...) * INTERVAL '1 day'`. 🗨️ **Gaps-and-islands:** subtracting a row counter from a date makes consecutive dates map to the same constant — group by it to find streaks. A top interview pattern.

94. Find the gaps — missing IDs in a sequence

```
-- Where the next order_id jumps by more than 1, there's a gap
SELECT order_id + 1 AS gap_start,
       next_id - 1 AS gap_end
FROM (
  SELECT order_id,
         LEAD(order_id) OVER (ORDER BY order_id) AS next_id
  FROM orders
) t
WHERE next_id - order_id > 1;
```

❄️ **Snowflake:** identical. 🗨️ `LEAD` looks at the next row; a jump > 1 reveals missing IDs.

95. Employees earning the same salary as someone else (duplicates via window)

```
SELECT name, salary
FROM (
  SELECT name, salary,
         COUNT(*) OVER (PARTITION BY salary) AS same_sal_count
  FROM employees
) t
WHERE same_sal_count > 1;
```

❄️ **Snowflake:** `... QUALIFY COUNT(*) OVER (PARTITION BY salary) > 1;` 🗨️ A partitioned `COUNT(*)` tags every row with how many share its salary — filter > 1 to find duplicates.

96. Rank customers by spend and keep only the top 3 (dense)

```

WITH spend AS (
  SELECT customer_id, SUM(amount) AS total
  FROM orders GROUP BY customer_id
)
SELECT customer_id, total,
  DENSE_RANK() OVER (ORDER BY total DESC) AS rnk
FROM spend
QUALIFY rnk <= 3; -- Snowflake

```

❄️ **Standard SQL:** wrap in a subquery and filter `WHERE rnk <= 3`.

97. Recursive CTE — the full management chain (org hierarchy)

```

-- Start at the CEO, then repeatedly join to find each person's reports
WITH RECURSIVE org AS (
  SELECT emp_id, name, manager_id, 1 AS level
  FROM employees
  WHERE manager_id IS NULL -- anchor: the top
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id, o.level + 1
  FROM employees e
  JOIN org o ON e.manager_id = o.emp_id -- recursive step
)
SELECT level, name FROM org ORDER BY level, name;

```

❄️ **Snowflake:** identical (`WITH RECURSIVE` supported). 🗨️ Recursion = anchor (starting rows) + recursive member (rows built from the previous step) until nothing new is added. The tool for trees/hierarchies.

98. Recursive CTE — generate a number series 1..10

```

WITH RECURSIVE nums AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM nums WHERE n < 10
)
SELECT n FROM nums;

```

❄️ **Snowflake:** identical. Useful for building calendars/date spines to fill gaps.

99. Fill missing months with zero revenue (date spine + LEFT JOIN)

```

WITH RECURSIVE months AS (
  SELECT DATE '2023-06-01' AS m
  UNION ALL
  SELECT DATEADD('month', 1, m) FROM months WHERE m < DATE '2023-08-01'
)
SELECT m,
  COALESCE(SUM(o.amount), 0) AS revenue
FROM months
LEFT JOIN orders o ON DATE_TRUNC('month', o.order_date) = m
GROUP BY m
ORDER BY m;

```

❄️ **Snowflake:** identical. 🗨️ Reporting must show months with **zero** activity — a generated date spine + `LEFT JOIN` + `COALESCE` does it. Common real-world requirement.

100. Year-over-year style growth % per month (LAG + calculation) — Capstone

```
WITH monthly AS (  
  SELECT DATE_TRUNC('month', order_date) AS mth, SUM(amount) AS revenue  
  FROM orders GROUP BY 1  
)  
SELECT mth, revenue,  
  LAG(revenue) OVER (ORDER BY mth) AS prev,  
  ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY mth))  
    / NULLIF(LAG(revenue) OVER (ORDER BY mth), 0), 1) AS growth_pct  
FROM monthly  
ORDER BY mth;
```

❄️ **Snowflake:** identical. 💬 `NULLIF(x, 0)` guards against divide-by-zero when the previous month is 0/NULL. This single query combines CTEs, window functions, and safe math — everything from this tier.

🎓 You Finished All 100!

You've now practised every pattern SQL interviews throw at you: joins, subqueries, CTEs, all window functions, ranking, running totals, LAG/LEAD, gaps-and-islands, pivots, and recursion — in both standard SQL and Snowflake.

Next steps: - Redo every problem you needed help on, one week later. - Try the same patterns on a dataset *you* care about. - Put "solved 100 SQL problems incl. window functions & recursion in SQL + Snowflake" on your resume.

🔙 Back to [Easy](#) · [Medium](#) · [Problem Index](#)

📖 SQL Mastery — [PJ's Academy](#) · hello@pjsacademy.com